
Universidad de Guayaquil
Facultad de Ciencias Matemáticas y Físicas
Carrera de Software

Informe del Sprint 2

Seguridad, administración y configuración inicial

Prototipo web de inteligencia de negocios asistido por IA

Autoras	Emily Dayan Robles Alvarado y Lesly Samay Velásquez Anchundia
Versión	0.1.0
Periodo de referencia	4 al 11 de septiembre de 2026
Rama de trabajo	<code>feature/rbac-y-parametros</code>
Línea base	<code>develop</code> ; promoción posterior mediante pull request
Repositorio	LFXAS/prototipo-bi-ia
Fecha	11 de septiembre de 2026

Índice

1. Resumen ejecutivo	1
2. Objetivo y alcance	1
2.1. Objetivo	1
2.2. Alcance entregado	1
2.3. Exclusiones	1
3. Desarrollo guiado por especificaciones	2
4. Arquitectura del incremento	2
4.1. Flujo de seguridad	2
4.2. Modelo RBAC	2
4.3. Controles de recuperación y eliminación	3
4.4. Evolución reproducible de base de datos	3
5. Experiencia de usuario	3
6. Parámetros generales	3
7. Configuración LLM	4
7.1. Principio de seguridad	4
7.2. Alternativa local verificable	4
8. Verificación y evidencia	5
9. Riesgos y controles	6
10.Cierre y siguientes pasos	6
11.Control documental	6

1. Resumen ejecutivo

El Sprint 2 transforma la línea base Docker del Sprint 1 en un incremento administrativo utilizable. Se implementa autenticación con JSON Web Token (JWT), autorización basada en roles y permisos (RBAC), gestión de usuarios, roles, permisos y menús, auditoría de acciones administrativas, parámetros operativos controlados y configuración segura de proveedores LLM.

El incremento no adelanta las capacidades analíticas del anteproyecto. Aún no implementa ETL, data-mart, exploración de metadatos, dashboard, reportería, predicción ni generación de propuestas BI. La configuración LLM prepara una integración controlada para un sprint posterior; no transmite datos de negocio ni ejecuta SQL.

2. Objetivo y alcance

2.1. Objetivo

Proveer una administración segura, comprensible y responsive que permita controlar quién accede al prototipo, qué acciones puede realizar y qué configuración LLM aprobada queda disponible para la fase posterior de Copiloto BI.

2.2. Alcance entregado

Capacidad	Entrega del incremento	Estado
Autenticación	Inicio y cierre de sesión, hash Argon2, JWT de vida limitada y cuenta administradora inicial desde el entorno.	Completado
RBAC	Usuarios, roles, permisos, asociaciones, autorización FastAPI y menús derivados de permisos.	Completado
Protección	Cuenta/rol base protegidos, invariantes de recuperación y validaciones de dependencias antes de eliminar.	Completado
Auditoría	Registro inmutable de inicio de sesión, creación, edición, estado, asociaciones, pruebas y eliminación.	Completado
UX administrativa	Navegación agrupada, plegable, ocultable, responsive, formularios guiados, tablas de casillas y paginación.	Completado
Parámetros y LLM	Catálogo sin claves libres, credenciales cloud cifradas desde la web, proveedores Gemini/Qwen/Ollama y prueba limitada desde backend.	Completado
Ollama local	Perfil Docker opcional, modelo ágil <code>qwen2.5:3b</code> persistente y prueba de disponibilidad real.	Completado

2.3. Exclusiones

Quedan fuera de este Sprint: recuperación de contraseña por correo, SSO, multiempresa, microservicios, tematización por empresa, ETL, datamart, KPIs, gráficos, alertas, reportería, forecasting y Copiloto BI funcional. Estas exclusiones son decisiones de alcance, no fallas del incremento.

3. Desarrollo guiado por especificaciones

Antes de implementar se organizaron los contratos SDD en `docs/specs/`. La especificación principal separa las responsabilidades para evitar pantallas mezcladas o reglas de seguridad implícitas.

Documento	Contrato que define	Evidencia vinculada
SPR-02-01-seguridad-rbac.md	Modelo RBAC, CRUD, protección administrativa, eliminación física validada y auditoría.	API, migración y pruebas.
SPR-02-02-navegacion-y-experiencia.md	Navegación responsive, agrupación, plegado, estados de pantalla, accesibilidad y controles legibles.	Prueba visual y frontend.
SPR-02-03-parametros-y-llm.md	Parámetros controlados, proveedores, credenciales web cifradas y prueba LLM segura.	Adaptadores, cifrado, prueba unitaria y prueba real.
SPR-02-04-identidad-visual-y-shell-bi.md	Tokens visuales y dirección para módulos futuros sin mostrarlos prematuramente.	CSS y revisión visual.
SPR-02-matriz-correcciones.md	Trazabilidad de observaciones funcionales y decisiones correctivas.	Revisión, pruebas y bitácora.

Nota. Una especificación no sustituye la revisión humana. Las autoras aprueban, corrigen o rechazan la propuesta antes de que pase a código, pruebas y pull request.

4. Arquitectura del incremento

4.1. Flujo de seguridad

React → FastAPI → JWT y permiso efectivo
 Si autoriza: **servicio RBAC/parámetros → PostgreSQL app**
 Cada mutación → **evento de auditoría inmutable**

React mejora la orientación de la persona usuaria y oculta acciones no autorizadas, pero FastAPI vuelve a comprobar JWT y permiso en cada endpoint. Así, manipular la interfaz no concede privilegios.

4.2. Modelo RBAC

Un usuario puede tener varios roles. Un rol agrupa permisos. Los permisos autorizan endpoints y determinan qué menús son visibles. Los códigos técnicos permanecen estables en backend/migraciones; la interfaz presenta nombres, descripciones y tablas de casillas para evitar que la persona escriba IDs o privilegios manualmente.

Las entidades de aplicación son `users`, `roles`, `permissions`, `user_roles`, `role_permissions`, `menus`, `menu_permissions`, `audit_events`, `parameters` y `llm_configurations`, bajo el esquema PostgreSQL app.

4.3. Controles de recuperación y eliminación

La cuenta y el rol administrativos iniciales poseen una marca persistida de protección. FastAPI rechaza desactivarlos, eliminarlos o dejarlos sin las capacidades mínimas de recuperación. La eliminación física de recursos no protegidos requiere confirmación, comprueba primero dependencias hijas y responde con conflicto explícito cuando deben resolverse asociaciones. Una eliminación permitida genera auditoría con acción **delete**; los eventos de auditoría no se modifican ni eliminan desde la interfaz.

La corrección post-cierre para cuentas temporales permite eliminar un usuario no protegido una vez retiradas sus asociaciones usuario-rol, incluso si fue actor de auditoría. La migración 20260911_05 usa **ON DELETE SET NULL** sobre la referencia técnica de actor y conserva una etiqueta histórica segura en el evento. La auditoría permanece disponible, mientras la cuenta temporal deja de existir.

4.4. Evolución reproducible de base de datos

El Sprint 2 formaliza la separación entre estructura, catálogo base y datos locales. Alembic aplica cambios de esquema versionados; inmediatamente después, el servicio Compose **seed** incorpora de manera idempotente permisos, menús, rol administrador y cuenta inicial protegidos. Así, cada programadora puede actualizar **develop** sin copiar un volumen PostgreSQL: su base conserva registros locales no protegidos y recibe el contrato técnico nuevo.

La semilla no transporta auditoría, cuentas temporales, claves, tokens, configuraciones LLM ni datos de negocio. Los datos demostrativos de módulos futuros deberán ser sintéticos, opcionales y aprobados mediante su especificación SDD.

5. Experiencia de usuario

La administración se integra en un cascarón responsive. En escritorio se puede contraer o restaurar el panel lateral mediante un chevrón compacto superpuesto al borde. En tableta y móvil se usa el patrón hamburguesa. Los grupos **Seguridad** y **Parámetros generales** son plegables incluso cuando contienen la ruta activa; Inicio queda como acceso directo.

Las listas reciben paginación desde FastAPI. Roles, usuarios y menús asignan relaciones mediante tablas de casillas con nombre y descripción. Al editar un usuario, la contraseña es opcional: si queda vacía se conserva la existente. Mensajes de éxito usan confirmación verde accesible; errores y validaciones se traducen a lenguaje comprensible y no muestran estructuras técnicas como `[object Object]`.

6. Parámetros generales

La pantalla Parámetros está preparada para configuraciones operativas aprobadas, pero no es una tabla libre de clave/valor. Un parámetro sólo podrá aparecer cuando exista un módulo consumidor, nombre humano, descripción, tipo, rango y validación definidos. En Sprint 2 no hay todavía un consumidor de parámetros operativos; por ello la pantalla muestra un estado vacío deliberado.

El primer catálogo se definirá con el módulo que lo necesite, por ejemplo ETL, reportería o analítica. Ese sprint deberá incluir la especificación SDD del valor, sus límites, formulario, auditoría y pruebas. No se guardarán contraseñas, tokens, cadenas de conexión ni valores arbitrarios como parámetros generales.

7. Configuración LLM

7.1. Principio de seguridad

La configuración conserva nombre, proveedor, URL permitida, modelo, estado y resultado seguro de la prueba. Para Gemini y Qwen Cloud, la persona administradora guarda primero la configuración y usa **Registrar credencial** o **Reemplazar credencial**. FastAPI cifra la clave; PostgreSQL conserva únicamente el texto cifrado en `app.secrets`, la raíz criptográfica permanece en el volumen Docker `secret_key_data` y React sólo recibe el estado **Credencial configurada**. Ollama local no requiere clave.

La migración 20260918_06 vincula cada configuración con, como máximo, un secreto. Cambiar de proveedor invalida la credencial anterior; eliminar una configuración inactiva elimina también su secreto. La auditoría registra alta o reemplazo sin incluir la clave. `GEMINI_API_KEY` y `DASHSCOPE_API_KEY` se retiraron de Compose y de la plantilla `.env.example`.

7.2. Alternativa local verificable

El perfil Compose `local-llm` incorpora Ollama de forma opcional y privada dentro de la red de contenedores. El modelo local predeterminado es `qwen2.5:3b`, elegido para mantener una experiencia ágil en equipos de desarrollo. `qwen3:4b` permanece como alternativa de mayor capacidad y mayor consumo. Se opera desde la raíz del repositorio:

```
make ollama-up
make ollama-pull
make ollama-status
make ollama-down
```

El modelo se guarda en el volumen `ollama_models`; cada programadora lo descarga una vez en su máquina. No se versiona con Git, no se publica en GHCR y CI/CD no lo descarga. La URL interna que debe registrarse en la pantalla es `http://ollama:11434`. FastAPI consulta `/api/tags` y sólo confirma éxito si el servicio responde y el modelo seleccionado está descargado. La verificación real del incremento confirmó esa condición sin enviar datos del negocio y sin alterar la configuración cloud activa.

8. Verificación y evidencia

Control	Resultado	Evidencia
Compose	Completado	Desarrollo, perfil <code>delivery</code> , perfil <code>local-llm</code> y entrega GHCR validados con configuración Compose.
Backend	Completado	Ruff, Mypy y 13 pruebas Pytest Docker; se cubren cifrado, manipulación del texto cifrado, credencial web, validación de URL y modelo Ollama disponible/no descargado.
Frontend	Completado	ESLint, 4 pruebas Vitest y construcción Vite dentro de Docker; se cubre el formulario de credencial de un solo uso.
LLM local	Completado	<code>qwen2.5:3b</code> predeterminado; prueba FastAPI verifica servicio y modelo interno.
Documentación	Completado	Bitácora, informe Sprint 1, informe Sprint 2 y manual técnico compilados reproduciblemente con LaTeX Docker.
Seguridad operativa	Completado	Prueba integrada con credencial temporal: cero coincidencias en texto plano, reutilización tras reiniciar y cero secretos huérfanos después de eliminar.

La comprobación integral se ejecuta con:

```
make verify
```

El comando no descarga modelos LLM: valida solamente que el perfil opcional sea una configuración Compose válida. Esta decisión evita que CI consuma almacenamiento, tiempo y recursos de cómputo en cada ejecución.

9. Riesgos y controles

Riesgo	Control aplicado	Resultado esperado
Pérdida de acceso administrativo	Marca protegida e invariantes verificadas en backend.	Siempre existe una vía de recuperación.
Exposición de secretos	Cifrado autenticado, raíz separada, respuesta segura y auditoría sin claves/JWT.	PostgreSQL sólo recibe texto cifrado y ninguna API key vuelve a React.
Borrado inconsistente	Confirmación, validación de hijos y respuesta 409; los eventos de auditoría conservan etiqueta histórica si se borra un actor temporal.	No hay borrado en cascada silencioso ni pérdida de trazabilidad.
Cuota cloud agotada	Ollama local opcional y modelo persistente.	Prueba LLM sin depender de cuota.
Uso excesivo de recursos	Perfil no automático, modelo explícito, límites de contexto y una sola carga.	El entorno normal no se hace más pesado.
Confusión entre módulos	Especificaciones separadas y estado aislado por ruta.	Formularios y mensajes corresponden al recurso activo.

10. Cierre y siguientes pasos

El incremento administrativo está construido y verificado localmente. Antes de declararlo cerrado en control de versiones, corresponde revisar los cambios, ejecutar `make verify`, abrir el pull request de `feature/rbac-y-parametros` hacia `develop`, obtener la aprobación requerida, fusionar y luego promover `develop` hacia `main` mediante un nuevo pull request.

Los siguientes sprints deben conservar este contrato: primero especificación SDD aprobada; después migraciones, API, interfaz, pruebas y documentación. El próximo módulo funcional recomendado es la introspección determinística de AdventureWorks y sus metadatos, no la generación automática de SQL por IA.

11. Control documental

Campo	Valor
Fuente editable	<code>docs/sprints/sprint-02-seguridad-y-parametros.tex</code>
Compilación	<code>make sprint2-report</code> o <code>make docs</code>
Documentos relacionados	Especificaciones SPR-02, ADR 0002, bitácora y manual técnico.
Revisión	Debe actualizarse si cambia la arquitectura, alcance, controles, pruebas o evidencia del Sprint 2.

Historial de cambios

Versión	Fecha	Cambio
0.1.0	11-09-2026	Primera versión: documentación del incremento administrativo, RBAC, UX, parámetros, proveedores LLM y alternativa local Ollama.
0.2.0	18-09-2026	Corrección final: registro web de credenciales cloud, almacenamiento cifrado, volumen de clave maestra, migración, auditoría y verificación integrada.